More Permutations and Combinations

Selection with Repetition

- The permutation and combination question we have done so far are basically about selecting objects.
  - You have n objects and select r of them. In how many ways if order does/doesn't matter?
  - Answers were $P(n, r)$ and $C(n, r)$.
  - … and we found problems where those were useful, but it wasn't obvious. (e.g. how many bitstrings with r ones?)
- Something in particular that was missing: being able to select with repetition.
- **Example:** How many distinct values can be represented with 5 decimal digits?
  - Here we are selecting items (digits) where repetition is allowed: we can select 4 multiple times if we want.
  - We can actually answer this with just the product rule: 10^5.
- That was an r-permutation of n items with repetition allowed.
  - Specifically, we select r objects from n possibilities, and are allowed to select the same object as many times as we want.
  - There are n^r different r-permutations of n items with repetition.
  - Proof: the product rule applied r times.

Combinations with Repetition

- We can also have an r-combination of n items with repetition.
  - Same as other combinations: order doesn't matter.
  - Same as permutations with repetition: we can select the same thing multiple times.
- **Example:** You walk into a candy store and have enough money for 6 pieces of candy. The store has chocolate (C), gummies (G), and Haribo sugar-free gummi-bears, which are horrible (H). How many different selections can you make?
  - Here are some possible selections you might make:

```
C C C G G H
C G G G G H
C C C C G G
H H H H H H
```

  - Since order doesn't matter, we'll list all of our selections in the same order: C then G then H.

- We don't want our candy to mix: let's separate the types.

```
C C C | G G | H
C | G G G G | H
C C C C | G G |
| | H H H H H H
```

- Now we don't need the actual identities in the diagram to know what's there:

```
- - - | - - | -
- | - - - - | -
- - - - | - - |
| | - - - - - -
```

- Now the answer becomes obvious: we have 8 slots there and just have to decide where to put the two dividers.
- There are $C(8,2)$ ways to do that, so $C(8,2) = 28$ possible selections.
- … or equivalently, there are $C(8,6) = 28$ ways to place the candy selections.

- If we are selecting an r-combination from n elements with repetition, there are $C(n+r-1,r) = C(n+r-1,n-1)$ ways to do so.
  - Proof: like with the candy, but not specific to $r = 6$ and $n = 3$.
- **Example:** How many solutions does this equation have in the non-negative integers?

$$a + b + c = 100$$

  - In order to satisfy the equation, we have to select 100 “ones”, some that will contribute to a, some to b, some to c.
  - In other words, we have balls labelled a, b, and c. We select 100 of them (with repetition) and that gives us a solution to the equation.
  - $C(102,2)$ solutions.
- In summary we have these ways to select r things from n possibilities:

Order?	Repetition?	Formula
Yes (permutation)	No	$P(n,r) = \frac{n!}{(n-r)!}$
No (combination)	No	$C(n,r) = \frac{n!}{r!(n-r)!}$
Yes (permutation)	Yes	n^r
No (combination)	Yes	$C(n+r-1,r) = \frac{(n+r-1)!}{r!(n-1)!}$

Permutations with Identical Objects

- There are many problems similar to the basic combination/permutation ones.
  - The basic approach is: how can you make it look like a problem you know how to solve? When you do, how much did over-/undercount by?
- **Example:** How many ways are there to permute the letters of the word HAPPY?
  - There would be $5!$ ways to permute HAPQY since all of the letters are different.

- How do we deal with the repeated P? Let's pretend they're different: $\mathrm{HAP_1P_2Y}$.
- Now there are $5!$ ways, but we counted both of these, but in the original problem they should only be counted once:

$$\mathrm{P_1AHP_2Y}$$
$$\mathrm{P_2AHP_1Y}$$

- In fact, we counted every permutation twice: with each possible ordering of the Ps.
- Real solution: $5!/2! = 60$.

- **Example (again):** How many ways are there to permute the letters of the word HAPPY?
  - Let's first decide where to put the Ps, in $C(5,2)$ ways.
  - Then in the remaining 3 positions, permute the remaining 3 elements.
  - Final answer:

$$C(5,2) \cdot P(3,3) = \frac{5!}{2!3!}\,\frac{3!}{0!} = 5!/2!\,.$$

- **Theorem:** There are $n!/k!$ ways to permute n objects where k are idential (but the other $n-k$ are different).

  Proof idea: Exactly as the previous example, with $n=5$ and $k=2$. [Second solution is much more of a proof. First is a little too sloppy to be “proof”, but is a good way to think about it.]

- **Theorem:** Suppose we have n items, where there are $n_1, n_2, \ldots, n_k$ that are identical. The number of ways to permute them is

$$\frac{n!}{n_1!n_2!\cdots n_k!}\,.$$

  We will demand that $\sum n_i = n$ for the proof, but obviously we don't really need to count one-of-a-kind items to get the right answer, since $1! = 1$.

  Proof: As before, first select positions for the n_1 identical items in $C(n,n_1)$ ways. Then place the n_2 items in $C(n-n_1,n_2)$ ways, and so on. The total number of ways to arrange the items is

$$\begin{aligned}
&C(n,n_1)C(n-n_1,n_2)C(n-n_1-n_2,n_3)\cdots C(n-n_1-\cdots-n_{k-1},n_k)\\
&= \tfrac{n!}{n_1!(n-n_1)!}\;\tfrac{(n-n_1)!}{n_2!(n-n_1-n_2)!}\;\tfrac{(n-n_1-n_2)!}{n_3!(n-n_1-n_2-n_3)!}\cdots\tfrac{(n-n_1-\cdots-n_{k-1})}{n_k!0!}\\
&= \frac{n!}{n_1!n_2!\cdots n_k!}\,. \qquad \blacksquare
\end{aligned}$$

- **Example:** In a card game with a single deck (no jokers), there are $52! \approx 8.1\times 10^{67}$ ways to order the deck. How many ways for a game that is played with two decks shuffled together?
  - There are 104 cards in total, but the pair of each card is identical (Two 2♠, two 2♥, …).
  - Thus, there are $\frac{104!}{(2!)^{52}} \approx 2.7\times 10^{167}$ ways to order the double-deck.

- [To give an idea of the scale of those numbers, there are around 3.6×10^{51} protons+neutrons making up the mass of the Earth.]

- **Example:** How many ways to order the letters of $\mathrm{MISSISSIPPI}$?
  - There are 11 letters, but four Is, four Ss, two Ps.
  - There are $\frac{11!}{4!4!2!} = 34650$ ways.

More Counting Examples

1. Your mother-in-law buys 1000 small gifts to give to relatives for Christmas, for reasons you don't understand. Each of the 1000 things is different, because she spends too much time shopping. There are 25 relatives to give gifts to. How many ways are there to distribute the gifts?
   - Note: the question allows the possibility of one person getting 1000 things, and everyone else getting nothing. There's no fairness in this family

2. In the above scenario, how many ways can the gifts be distributed so each person gets 40 items?

3. The next year, you are sent to the basement to arrange the 1000 gifts into 25 piles, with one for each person. You don't know who will be getting each pile, so the order of the piles doesn't matter. How many arrangements are there for you to choose from?

4. The next year, your mother-in-law buys 1000 identical pens to give to the family. You start to wonder what is going on. How many ways can they be distributed to 25 family members?

5. How many ways if each person must get at least 3 pens?

6. How many ways if each person gets 40 pens?

Solutions

1. This is a permutation with repetition. For each gift, it can be given to one of 25 family members. First gift in 25 ways, second gift in 25 ways, …. In total, $25^{1000} \approx 8.7 \times 10^{1397}$ ways.

2. Think of giving each family member 40 tokens with their name on it. Line the 1000 different gifts up in a row and have each family member drop their 40 tokens on gifts.

   A final arrangement in this case is a permutation of the 1000 tokens, with 25×40 identical ones. It is a permutation of identical objects as above and the number of permutations is

$$\frac{1000!}{(40!)^{25}} \approx 5.3 \times 10^{1369}\,.$$

3. This one is surprisingly difficult. See the textbook's discussion of “distinguishable objects and indistinguishable boxes” on p. 337, or look up Stirling Numbers of the second kind.

4. To distribute the gifts, we must select people to get each one. Order doesn't matter here, since the pens are identical. This is a combination with repetition problem: combinations of 1000 the 25 family members with repetition. So there are $C(1024, 25)$ ways to distribute.

5. Attempt #1: Count all $C(1024, 25)$ ways to distribute the pens, then subtract the number where somebody gets 2 or fewer pens. How many of those are there? Ummmm…

   Attempt #2: First give everybody three pens, using 75 of them. Then distribute the remaining 925 in $C(949, 25)$ ways.

6. One way: give each person 40 pens.

Generating Permutations and Combinations

- It's occasionally useful to actually generate all of the permutations/combinations, not just count them.
  - e.g. you want to do write code to search for a perm/comb that satisfies some condition.
  - e.g. you want to write an automated test for code that should be able to handle input in any order.
- We should first decide on a way to order permutations. Then we can generate them in that order, and be sure we found them all.
- We will order them based on lexicographic order.
  - … which is a fancy way to say “dictionary order”.
- We will say that a permutation $a_1 a_2 \ldots a_n$ is less than $b_1 b_2 \ldots b_n$ if either:
  - a_1 is less than b_1.
  - $a_1 = b_1$ and $a_2 \ldots a_n$ is less than $b_2 \ldots b_n$.
- And obviously:
  - Two permutations are equal if all elements are the same.
  - A permutation A is greater than B if B is less than A.
- For example, if we are permuting the letters $ABCDE$, then the permutation $ABDEC$ is less than $ABEDC$.
- Aside: lexicographic ordering comes up in weird places. Sometimes you just need to know that some objects can be ordered so a proof/algorithm works out. Often you can just say “lexicographic ordering” and you're done.
- The following algorithm will generate all permutations of elements of a set, in lexicographic order:

```
procedure all_permutations(S)
   if length(S) == 1
      return the element as a length-one permutation
   else
      all_perm = []
      for each x in S.in_sorted_order
         S1 = S - {x}
         for each P in all_permutations(S1)
            all_perm += [x] + P
      return all_perm
```

- It's probably easiest to see how that works if we unwind the recursion a few steps.
  - For a one-element set, {1}, it returns that element in its only order: [1]
  - For a two-element set, {1,2} it recurses…
    - With x=1 on {2}, getting one result for all_perm: [1,2].
    - With x=2 on {1}, getting one result for all_perm: [2,1].
  - For a three element set, {1,2,3} it recurses…
    - With x=1 on {2,3}, getting [1,2,3] and [1,3,2].
    - With x=2 on {1,3}, getting [2,1,3] and [2,3,1].
    - With x=3 on {1,2}, getting [3,1,2] and [3,2,1].
- Can you convince yourself this generates $n!$ permutations? Easy enough to prove by induction.
- To generate all combinations of a set we make this observation: take a parcular element $x \in S$. The combination of S are…
  - All combinations of $S - \{x\}$.
  - All combinations of $S - \{x\}$ with x added in.
- The following algorithm will generate all combinations of elements of a set:

```
procedure all_combinations(S)
   if length(S) == 0
      return {}
   else
      all_comb = {}
      x = some element of S
      Sx = S-{x}
      for each C in all_combinations(Sx)
         all_comb += C
         all_comb += {x} ∪ C
      return all_comb
```

- For the set {1,2,3}, this algorithm does…
  - all_combinations({2,3})
  - all_combinations({3})
  - all_combinations({}), which returns {}
  - all_combinations({3}) returns {{}, {3}}
  - all_combinations({2,3}) returns {{}, {2}, {3}, {2,3}}
  - all_combinations({1,2,3}) returns {{}, {1}, {2}, {1,2}, {3}, {1,3}, {2,3}, {1,2,3}}
- Can you convince yourself this generates 2^n combinations? Easy enough to prove by induction.
- An example of recursion making things easier: compare the text's non-recursive algorithms to do these things. Which is more clear? Which one do you believe is correct?

- Actually the text's algorithms generate the next perm/comb and must be iterated to generate all of them.
- I have had to do that zero times in my life. I have had to generate all of them.